Whitebox Testing Report for MoneyPoly

Lohith Kola

March 23, 2026

1 Introduction

In this part of the assignment, I focused only on the whitebox side of the MoneyPoly codebase. I first cleaned the code with `pylint`, then wrote deterministic `pytest` test cases based on the internal branches, state changes, and edge cases inside the code. After that, I ran the tests repeatedly, identified the defects that were actually in the source code, fixed them one by one, and kept the full history as separate commits.

2 Codebase Overview

The MoneyPoly program is a small command-line board game. The main gameplay logic sits in `game.py`, while the other modules split out state and helper behavior:

- `bank.py`: manages bank funds, payouts, and emergency loans.
- `board.py`: stores the board layout and tile lookup logic.
- `cards.py`: defines Chance and Community Chest decks.
- `dice.py`: handles dice rolls, totals, and doubles tracking.
- `player.py`: stores player position, money, jail state, and owned properties.
- `property.py`: stores property data, mortgage state, and color-group ownership checks.
- `ui.py`: contains console I/O helpers.
- `main.py`: acts as the entry point.

From a whitebox testing point of view, the main risk areas were game flow, money transfer rules, jail handling, ownership transitions, and state changes that depend on internal branching.

3 Control Flow Graph

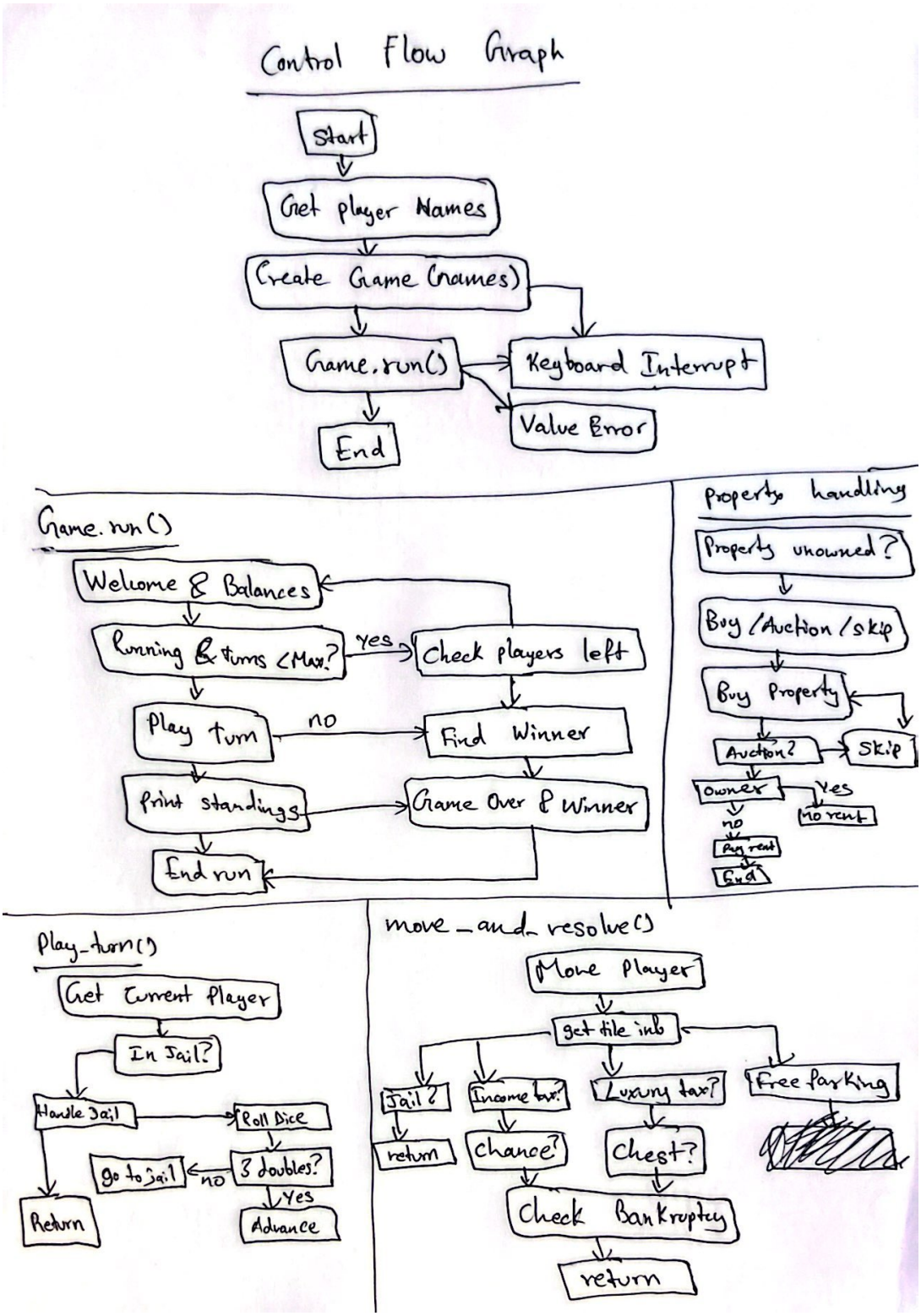

Figure 1: Control flow graph for the MoneyPoly system

4 Pylint Code Quality Analysis

4.1 Approach

I ran `pylint` on all required source files inside the whitebox MoneyPoly folder and fixed the results iteratively instead of making one large unsafe refactor. I prioritized problems that could affect readability or reliability, such as unused imports, dead code, broad exceptions, missing final newlines, long literals, and misleading comparisons. For a few model-heavy classes such as `Game`, `Player`, and `Property`, I used narrow `pylint` suppressions for complexity-related warnings instead of forcing artificial refactors that would have made the code harder to follow.

After the cleanup phase, the final pylint score for the required source set was **10.00/10**.

4.2 Main Warning Categories I Fixed

- Missing module, class, and function docstrings in the entry point and core modules.
- Unused imports and dead local variables.
- Line-length issues in the card tables.
- Overly broad exception handling in the UI helper.
- Superfluous comparisons and unnecessary control-flow branches.
- Missing final newline and minor formatting issues.
- Attribute initialization issues in the dice model.

4.3 Pylint Iterations

Iteration	What I changed
1	Moved the extracted MoneyPoly source into `whitebox/code/moneypoly` and added the minimal package/test bootstrap needed for whitebox execution.
2	Added module and function docstrings in `main.py`.
3	Removed an unused import and documented the bank module and class.
4	Added a board module docstring and simplified the mortgaged-property check.

Iteration	What I changed
5	Reformatted the card tables into readable multi-line literals and documented the cards module.
6	Added a module docstring to the shared configuration file.
7	Initialized dice state cleanly in `__init__` and removed an unused import.
8	Cleaned the game module lint issues, removed unused imports, fixed small control-flow warnings, and used narrow pylint suppressions for unavoidable complexity warnings.
9	Removed dead code from the player model and documented the module.
10	Documented the property models and simplified the mortgage return path.
11	Documented the UI module and replaced the bare `except` with explicit exception handling.

5 White-Box Test Design

5.1 Testing Strategy

I wrote the tests with the source structure in mind instead of treating the program like a black box. I targeted three things in every module:

- decision branches, such as unowned vs owned properties, valid vs invalid payments, jail choices, and doubles handling,
- important variable states, such as balances, mortgage flags, ownership lists, jail state, turn index, and dice streaks,
- edge cases, such as empty card decks, exact-balance purchases, passing Go, and bankrupt-player removal.

I also avoided fragile console-format tests. Whenever randomness or interactive input was involved, I used `monkeypatch` so the tests stayed deterministic.

5.2 Test Cases and Why They Matter

Bank tests

- `test_collect_and_pay_out_update_bank_balance`: I wrote this to check the normal cash-in and cash-out path in the bank.
- `test_pay_out_rejects_insufficient_funds`: I needed this branch because the bank should fail loudly instead of silently going negative.
- `test_give_loan_credits_player_and_reduces_bank_reserves`: I wrote this to verify the emergency-loan branch actually transfers funds. This test exposed a real defect.

Board tests

- `test_get_tile_type_handles_special_property_and_blank_tiles`: I used this to cover the three main tile lookup outcomes.
- `test_is_purchasable_requires_unowned_and_unmortgaged_property`: I needed this to force the ownership and mortgage branches.
- `test_board_ownership_helpers_track_owner_changes`: I wrote this to check that helper queries reflect state changes correctly.

Card tests

- `test_draw_cycles_and_peek_preserves_order`: I used this to verify deck cycling and non-advancing peek behavior.
- `test_cards_remaining_and_repr_handle_empty_deck`: I wrote this to hit the empty-deck edge case. It exposed a real crash.
- `test_reshuffle_resets_index`: I used this to verify the reshuffle branch resets the draw pointer properly.

Dice tests

- `test_roll_uses_standard_six_sided_bounds`: I wrote this because dice bounds are easy to get wrong and affect the whole game. This test exposed a real defect.
- `test_doubles_streak_increments_and_resets`: I needed this to cover both the doubles and non-doubles branches in the same state machine.
- `test_describe_marks_doubles_and_shows_total`: I used this to verify the last-roll state is represented consistently.

Game flow tests

- `test_handle_jail_turn_using_card_releases_player_and_moves`: I wrote this to exercise the jail-free-card branch.
- `test_handle_jail_turn_paying_fine_deducts_player_balance`: I needed this because the fine-payment branch changes two balances and a jail state. It exposed a real defect.
- `test_apply_card_move_to_resolves_destination_property`: I used this to verify that a movement card continues into tile resolution.
- `test_apply_card_birthday_collects_only_from_players_with_enough_cash`: I wrote this to cover the selective transfer branch in the birthday card logic.
- `test_move_and_resolve_sends_player_to_jail`: I needed this for the go-to-jail tile branch.
- `test_move_and_resolve_handles_railroad_property_branch`: I used this to force the railroad branch in the resolver.
- `test_find_winner_returns_highest_net_worth_player`: I wrote this because winner selection is a final-state decision that must be correct. It exposed a real defect.
- `test_play_turn_after_bankruptcy_advances_to_next_survivor`: I used this to check a subtle turn-order edge case after elimination. It exposed a real defect.
- `test_play_turn_keeps_same_player_after_non_terminal_doubles`: I needed this to verify the extra-turn branch.
- `test_play_turn_sends_player_to_jail_after_third_doubles`: I wrote this to cover the three-doubles punishment branch.
- `test_apply_card_collects_go_salary_when_wrapping`: I used this to verify the wraparound branch in movement-card handling.

Game transaction tests

- `test_game_requires_at_least_two_players`: I wrote this because the setup path already suggested a minimum-player rule. It exposed a real defect.
- `test_buy_property_allows_exact_balance_purchase`: I needed this exact-value edge case because boundary checks often go wrong. It exposed a real defect.
- `test_pay_rent_transfers_money_to_owner`: I wrote this to verify both sides of a rent transfer. It exposed a real defect.

- `test_trade_moves_cash_to_seller_and_transfers_property`: I used this to check that a successful trade updates both ownership and balances. It exposed a real defect.
- `test_unmortgage_keeps_property_mortgaged_when_owner_cannot_pay`: I wrote this to test the failed-unmortgage edge case. It exposed a real defect.
- `test_auction_ignores_invalid_bids_and_awards_highest_valid_bid`: I needed this to cover low bid, unaffordable bid, and valid winner branches in the auction loop.

Player tests

- `test_money_operations_reject_negative_amounts`: I wrote this to verify the argument-validation branches in balance updates.
- `test_move_collects_go_salary_when_passing_go`: I used this to check the wraparound path. It exposed a real defect.
- `test_move_collects_go_salary_when_landing_on_go`: I wrote this to verify the direct-landing branch.
- `test_net_worth_includes_owned_property_values`: I used this because rankings and standings depend on net worth. It exposed a real defect.
- `test_go_to_jail_updates_position_and_flags`: I wrote this to verify jail-state reset behavior.

Property tests

- `test_group_ownership_requires_full_set_for_rent_bonus`: I wrote this to make sure the rent bonus only applies when the full set is owned. It exposed a real defect.
- `test_complete_group_doubles_rent`: I used this to verify the intended happy path after the full-set check.
- `test_mortgage_and_unmortgage_toggle_state_and_cost`: I wrote this to cover the mortgage/unmortgage transitions and returned amounts.
- `test_owner_counts_aggregate_by_player`: I used this to verify the property-group ownership summary branch.

6 Test Execution Summary

My first full test run ended with **24 passing tests and 14 failing tests**. Those failures were useful because they were tied to genuine logic problems in the program rather than test setup mistakes.

After fixing the defects one by one, the final test run finished with:

- **38 tests passed**
- **0 tests failed**

This gave me confidence that the main whitebox branches and state transitions in the MoneyPoly implementation were behaving correctly after the fixes.

7 Errors Found and Fixes Applied

Error	Problem found	How I fixed it
1	Rent was doubling even when a player owned only part of a color group.	I changed the group-ownership check from an `any()` style condition to a true full-set check using `all()`.
2	Emergency loans increased the player's money but did not decrease bank reserves.	I made the loan path use the bank payout logic so the money now leaves the bank correctly.
3	Empty card decks crashed when `cards_remaining()` or `__repr__()` was called.	I added explicit empty-deck handling that returns safe zero-state values.
4	Dice rolls only used values from 1 to 5.	I corrected both dice to roll over the full 1 to 6 range.
5	A player could choose to pay the jail fine without actually losing money.	I deducted the fine from the player before releasing them from jail.
6	Winner selection picked the player with the lowest net worth.	I changed the winner lookup from `min()` to `max()`.
7	Eliminating the current player could skip the next active player in turn order.	I adjusted `current_index` when removing a bankrupt player so the turn rotation stays correct.
8	The game allowed a one-player session even though setup expected at least two players.	I added a validation check in the game constructor and now raise `ValueError` for fewer than two players.
9	Buying a property failed when the player had exactly the asking price.	I fixed the affordability boundary check so equal balance is allowed.
10	Rent was deducted from the visiting player but never added to the owner.	I credited the property owner during rent transfer.
11	Trades removed money from the buyer but did not pay the seller.	I added the missing cash credit to the seller during a successful trade.
12	A failed unmortgage attempt cleared the mortgage flag anyway.	I moved the affordability check before the state change so the property stays mortgaged if payment fails.
13	Passing Go without landing exactly on it did not award salary.	I updated movement logic to pay salary on wraparound as well as direct landing.
14	Net worth ignored the value of owned properties.	I updated the net-worth calculation to include property value, with mortgaged properties counted at mortgage value.

Table 2: Errors found during whitebox testing and the fixes applied

8 Final Reflection

This whitebox phase was useful because several of the defects were not obvious from casual play. The most interesting issues were the ones caused by internal state transitions rather than visible syntax problems, especially the skipped turn after bankruptcy, the incorrect winner selection, the exact-balance purchase bug, and the rent/loan/trade money-transfer mistakes. Those are the kinds of errors that are easy to miss if I only test normal paths.

I also found that writing deterministic tests with patched input and randomness made the debugging process much faster. Once the branches were isolated properly, the failures were clear and each fix could be committed cleanly on its own.

9 Reproducibility

I kept the whitebox execution steps simple:

- install `pytest` and `pylint`,
- run `pylint` on the MoneyPoly files inside `whitebox/code/moneypoly`,
- run `python3 -m pytest whitebox/tests -q`.

I also added these commands to `whitebox/README.md` so the whitebox portion is ready to run again without extra setup work.